

COMPUTER ORGANIZATION AND ASSEMBLY LANGUAGE ASSIGNMENT#3



Name: Muhammad Annas Raza

Aetesam Asfar Ullah

Enrollment: 02-134232-061

02-134232-092

Class Section: BSCS 4B

Semester: Spring 2025

DEPARTMENT OF COMPUTER SCIENCE
BAHRIA UNIVERSITY, KARACHI CAMPUS, PAKISTAN

Solution:

New ISA Instruction Format Design

The new ISA structure focuses on efficient encoding while preserving flexibility. The proposed instruction format is:

Field	Size (bits)	Description
Opcode	8 bits	Operation Code (identifies the instruction)
Operand 1	8 bits	Register or Memory Address
Operand 2	8 bits	Register, Immediate Value, or Address
Addressing Mode 1	4 bits	Specifies how Operand 1 is accessed
Addressing Mode 2	4 bits	Specifies how Operand 2 is accessed
Reserved Byte	8 bits	Reserved for future use (must remain)
Miscellaneous	6 bits	Extra control bits
Total	64 bits	

Field Explanation:

- **Opcode (8 bits):**
 - 256 possible instructions — more than enough for our needs.
- **Operand 1 (8 bits):**
 - Can store register number or memory address.
- **Operand 2 (8 bits):**
 - Immediate value, another register, or memory address.
- **Addressing Modes (4 bits each):**
 - 16 different addressing modes possible.
- **Reserved Byte and Miscellaneous:**
 - **Not to be changed.**

Addressing Modes Used:

Three addressing modes are used in the newly designed ISA:

- **Register Direct Addressing Mode (Code: 0001):**

Operands are located inside CPU registers, allowing faster execution.

- **Immediate Addressing Mode (Code: 0010):**

The operand is a constant value given directly in the instruction.

- **Direct Memory Addressing Mode (Code: 0011):**

The instruction directly specifies the memory address where the operand is stored.

Instruction Operations and Their Addressing Modes

the operations supported by the new ISA and the addressing modes assigned to them are described below:

1. ADD (Addition) Operation

The ADD (Addition) operation uses the Register Direct Addressing Mode. An example instruction is ADD AX, BX, which adds the contents of BX to AX.

Assembly Code:

```
ORG 100h

MOV AX, 5
MOV BX, 10
ADD AX, BX

RET
```

Output:

The screenshot shows an x86 emulator window titled 'emulator: noname.com_'. The assembly code window displays the following instructions:

```
01 ORG 100h
02
03 MOV AX, 5
04 MOV BX, 10
05 ADD AX, BX
06
07 RET
```

The registers window shows the following values:

Register	H	L
AX	00	0F
BX	00	0A
CX	00	09
DX	00	00
CS	F400	
IP	0154	
SS	0700	
SP	FFFA	
BP	0000	
SI	0000	
DI	0000	

The memory window shows the following data:

Address	Value	Comment
F4150:	FF	255 RES
F4151:	FF	255 RES
F4152:	CD	205 =
F4153:	20	032 SPA
F4154:	CF	207 ±
F4155:	00	000 NULL
F4156:	00	000 NULL
F4157:	00	000 NULL
F4158:	00	000 NULL
F4159:	00	000 NULL
F415A:	00	000 NULL
F415B:	00	000 NULL
F415C:	00	000 NULL
F415D:	00	000 NULL
F415E:	00	000 NULL
F415F:	00	000 NULL
F4160:	FF	255 RES
F4161:	FF	255 RES
F4162:	CD	205 =
F4163:	1A	026 →
F4164:	CF	207 ±

The BIOS window shows the following data:

Address	Value	Comment
BIOS DI	INT 020h	
IRET		
ADD	IBX + SI, AL	
ADD	IBX + SI, AL	
ADD	IBX + SI, AL	
ADD	IBX + SI, AL	
ADD	IBX + SI, AL	
ADD	BH, BH	
DEC	BP	
SBB	CL, BH	
ADD	IBX + SI, AL	
ADD	IBX + SI, AL	
ADD	IBX + SI, AL	
ADD	IBX + SI, AL	
ADD	IBX + SI, AL	
ADD	BH, BH	
DEC	BP	
ADD	IBX + SI, AL	
ADD	IBX + SI, AL	

The extended value viewer window shows the value of AX in various formats:

Format	H	L
hex	00	0F
bin	00000000	00001111
oct	000	017
decimal 8 bit		
unsigned	0	15
signed	0	15
ascii		*
decimal 16 bit		
unsigned	15	
signed	15	

After execution, AX = 15.

2. MUL (Multiplication) Operation

The MUL (Multiplication) operation uses the Register Direct Addressing Mode. The instruction MUL BX multiplies the contents of BX with AX.

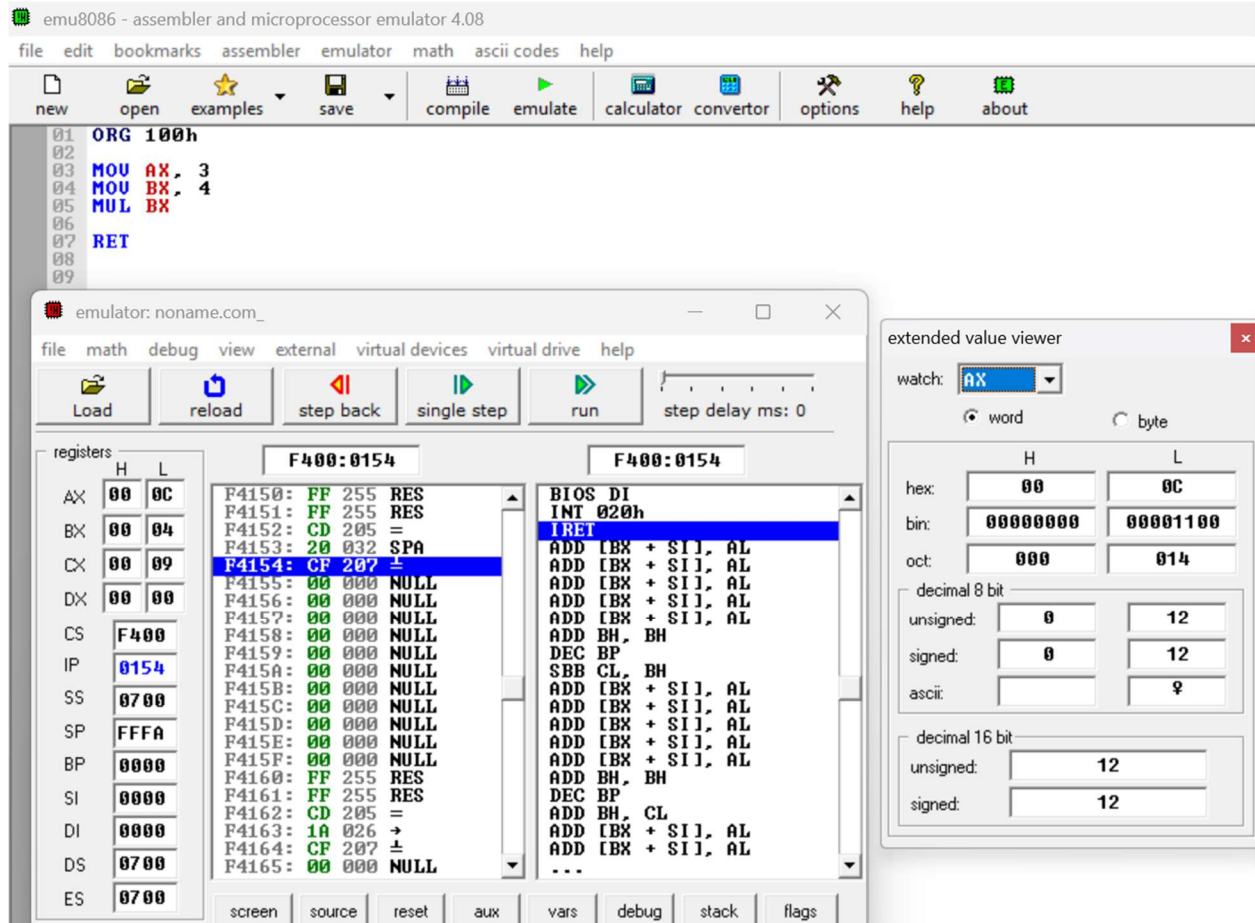
Assembly Code:

```
ORG 100h

MOV AX, 3
MOV BX, 4
MUL BX

RET
```

Output:



After execution, AX = 12.

3. AND (Logical AND) Operation

The AND (Logical AND) operation uses Register Direct Addressing Mode. The instruction AND AX, BX performs a bitwise AND between AX and BX.

Assembly Code:

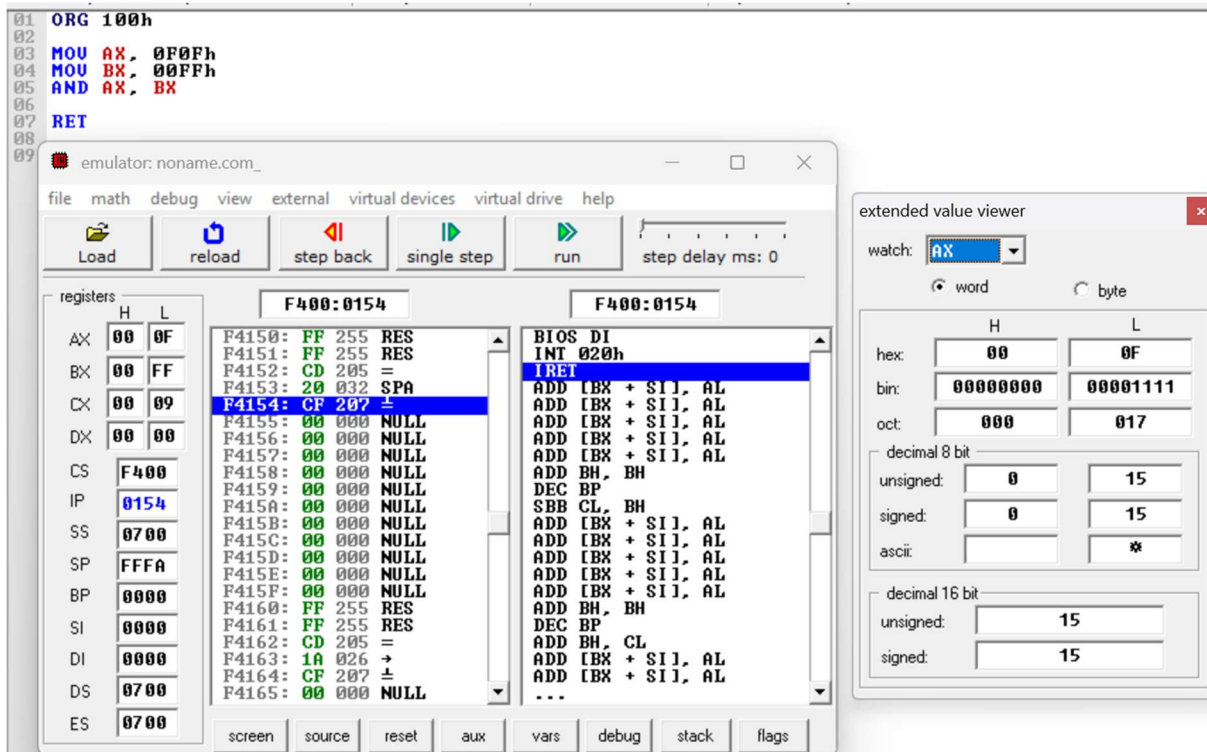
```

ORG 100h

MOV AX, 0F0Fh
MOV BX, 00FFh
AND AX, BX

RET
      
```

Output:



After execution, AX = 000Fh.

4. XOR (Logical XOR) Operation

The XOR (Logical XOR) operation uses Register Direct Addressing Mode. The instruction XOR AX, BX performs a bitwise XOR between AX and BX.

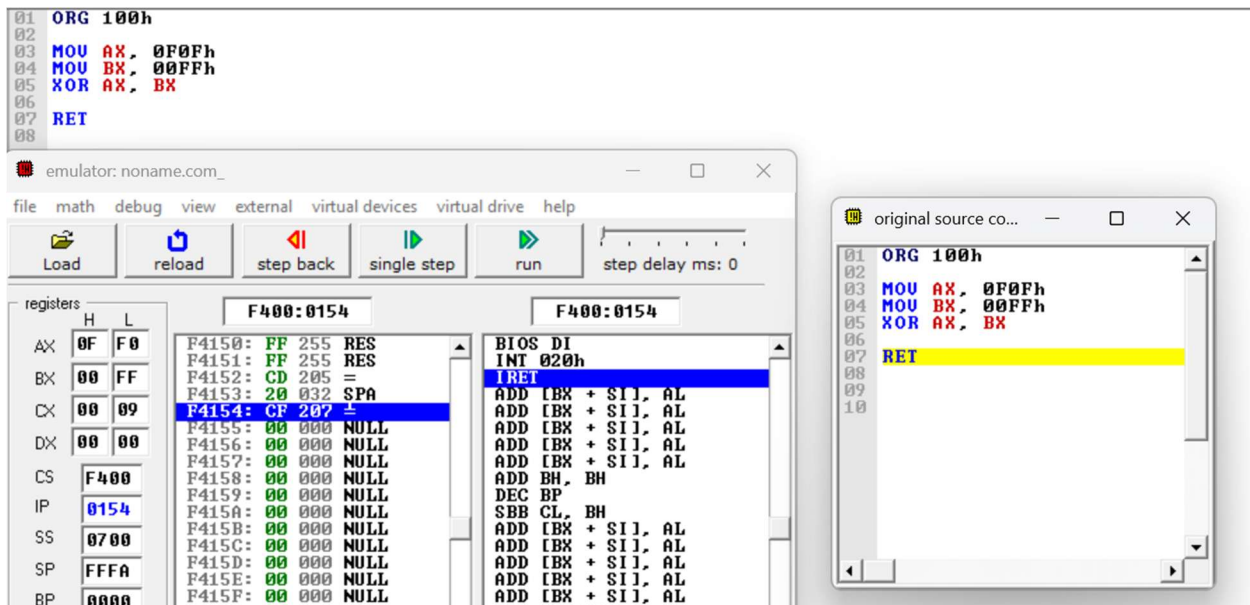
Assembly Code:

```
ORG 100h

MOV AX, 0F0Fh
MOV BX, 00FFh
XOR AX, BX

RET
```

Output:



5. EQ (Equal Comparison) Operation

The EQ (Equal Comparison) operation uses Register Direct Addressing Mode. It uses CMP AX, BX followed by JE label to jump if AX equals BX.

Assembly Code:

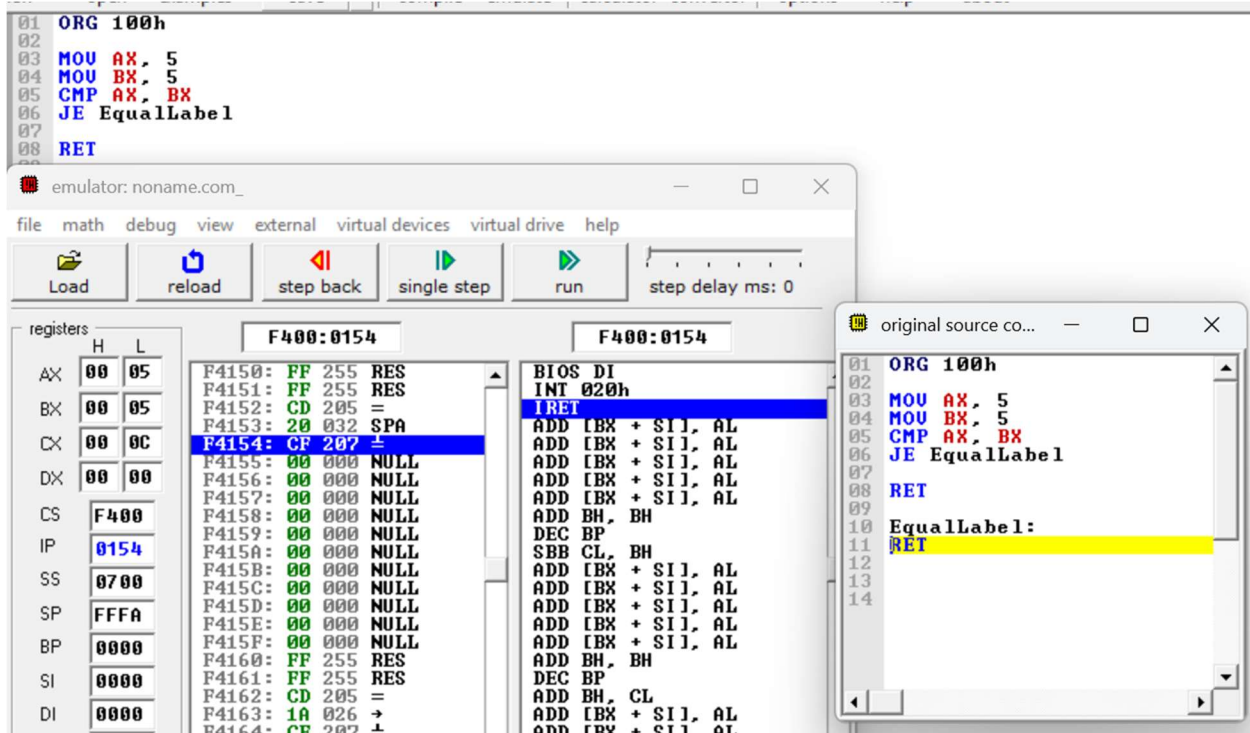
```
ORG 100h

MOV AX, 5
MOV BX, 5
CMP AX, BX
JE EqualLabel

RET

EqualLabel:
RET
```

Output:



The program jumps to EqualLabel if AX equals BX.

6. NE (Not Equal Comparison) Operation

The NE (Not Equal Comparison) operation uses Register Direct Addressing Mode. It uses CMP AX, BX followed by JNE label to jump if AX does not equal BX.

Assembly Code:

```
ORG 100h

MOV AX, 7
MOV BX, 9
CMP AX, BX
JNE NotEqualLabel

RET

NotEqualLabel:
RET
```


Output:

The screenshot shows an x86 emulator window titled 'emulator: noname.com_'. The assembly code window displays the following code:

```
01 ORG 100h
02
03 MOV AX, 7
04 MOV BX, 9
05 CMP AX, BX
06 JNE NotEqualLabel
07
08 RET
```

The registers window shows the following values:

Register	H	L
AX	00	07
BX	00	09
CX	00	0C
DX	00	00
CS	F400	
IP	0154	
SS	0700	
SP	FFFA	
BP	0000	
SI	0000	
DI	0000	

The memory window shows the following data:

Address	Value	Comment
F4150:	FF	255 RES
F4151:	FF	255 RES
F4152:	CD	205 =
F4153:	20	032 \$PA
F4154:	CF	207 ±
F4155:	00	000 NULL
F4156:	00	000 NULL
F4157:	00	000 NULL
F4158:	00	000 NULL
F4159:	00	000 NULL
F415A:	00	000 NULL
F415B:	00	000 NULL
F415C:	00	000 NULL
F415D:	00	000 NULL
F415E:	00	000 NULL
F415F:	00	000 NULL
F4160:	FF	255 RES
F4161:	FF	255 RES
F4162:	CD	205 =
F4163:	1A	026 →

The BIOS window shows the following data:

Address	Value	Comment
BIOS DI	INT 020h	
0000:	INT 020h	
0001:	ADD [BX + SI], AL	
0002:	ADD [BX + SI], AL	
0003:	ADD [BX + SI], AL	
0004:	ADD [BX + SI], AL	
0005:	ADD [BX + SI], AL	
0006:	ADD [BX + SI], AL	
0007:	DEC BH, BH	
0008:	DEC BP	
0009:	SBB CL, BH	
000A:	ADD [BX + SI], AL	
000B:	ADD [BX + SI], AL	
000C:	ADD [BX + SI], AL	
000D:	ADD [BX + SI], AL	
000E:	ADD [BX + SI], AL	
000F:	ADD [BX + SI], AL	
0010:	ADD BH, BH	
0011:	DEC BP	
0012:	ADD BH, CL	
0013:	ADD [BX + SI], AL	

The original source code window shows the following code:

```
1 ORG 100h
2
3 MOV AX, 7
4 MOV BX, 9
5 CMP AX, BX
6 JNE NotEqualLabel
7
8 RET
9
10 NotEqualLabel:
11 RET
```

The program jumps to NotEqualLabel if AX is not equal to BX.

7. ROTR (Rotate Right) Operation

The ROTR (Rotate Right) operation uses Register Direct Addressing Mode. The instruction ROR AX, 1 rotates the contents of AX one bit to the right.

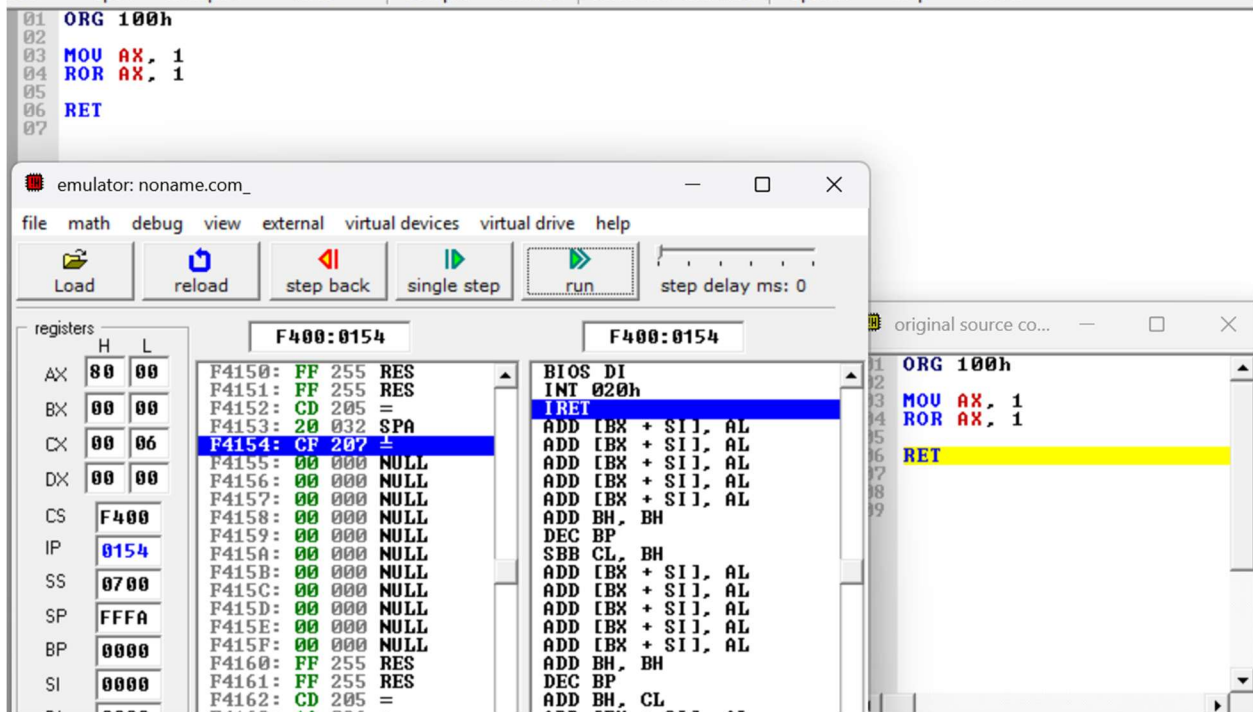
Assembly Code:

```
ORG 100h

MOV AX, 1
ROR AX, 1

RET
```

Output:



After execution, AX is rotated right by 1 bit.

8. MOV (Move Data) Operation

The MOV (Move Data) operation uses Immediate Addressing Mode or Register Direct Addressing Mode. Examples include MOV AX, 10 to load a constant or MOV AX, BX to copy between registers.

Assembly Code:

```
ORG 100h

MOV AX, 20
MOV BX, AX

RET
```

Output:

Assembly code in the emulator:

```

01 ORG 100h
02
03 MOV AX, 20
04 MOV BX, AX
05
06 RET
07

```

Registers window (F400:0154):

Register	H	L
AX	00	14
BX	00	14
CX	00	06
DX	00	00
CS	F400	
IP	0154	
SS	0700	
SP	FFFA	
BP	0000	
SI	0000	
DI	0000	
DS	0700	

Extended Value Viewer (watch: BX):

	H	L
hex:	00	14
bin:	00000000	00010100
oct:	000	024
decimal 8 bit		
unsigned:	0	20
signed:	0	20
ascii:		9
decimal 16 bit		
unsigned:	20	
signed:	20	

After execution, BX = 20.

9. BR (Branch) Operation

The BR (Branch) operation uses Direct Memory Addressing Mode. It uses the CALL address instruction to call a subroutine.

Assembly Code:

```

ORG 100h

CALL Subroutine

RET

Subroutine:
MOV AX, 1234h
RET

```

Output:

The screenshot shows an 8086 emulator window titled "emulator: noname.com_". The assembly code is as follows:

```
01 ORG 100h
02
03 CALL Subroutine
04
05 RET
06
07 Subroutine:
08 MOV AX, 1234h
09 RET
```

The registers window shows the following values:

Register	H	L
AX	12	34
BX	00	00
CX	00	08
DX	00	00
CS	F400	
IP	0154	
SS	0700	
SP	FFFA	
BP	0000	
SI	0000	
DI	0000	
DS	0700	

The memory window shows the following data:

Address	Value	Comment
F4150:	FF 255	RES
F4151:	FF 255	RES
F4152:	CD 205	=
F4153:	20 032	SPA
F4154:	CF 207	±
F4155:	00 000	NULL
F4156:	00 000	NULL
F4157:	00 000	NULL
F4158:	00 000	NULL
F4159:	00 000	NULL
F415A:	00 000	NULL
F415B:	00 000	NULL
F415C:	00 000	NULL
F415D:	00 000	NULL
F415E:	00 000	NULL
F415F:	00 000	NULL
F4160:	FF 255	RES
F4161:	FF 255	RES
F4162:	CD 205	=
F4163:	1A 026	→
F4164:	CF 207	±
F4165:	00 000	NULL

The BIOS window shows the following data:

Address	Value	Comment
BIOS DI	INT 020h	
I RET		
ADD [BX + SI], AL		
ADD [BX + SI], AL		
ADD [BX + SI], AL		
ADD [BX + SI], AL		
ADD [BX + SI], AL		
ADD BH, BH		
DEC BP		
SBB CL, BH		
ADD [BX + SI], AL		
ADD [BX + SI], AL		
ADD [BX + SI], AL		
ADD [BX + SI], AL		
ADD [BX + SI], AL		
ADD BH, BH		
DEC BP		
ADD BH, CL		
ADD [BX + SI], AL		
ADD [BX + SI], AL		
...		

The program calls the Subroutine and AX is set to 1234h.

10. JMP (Jump) Operation

The JMP (Jump) operation uses Direct Memory Addressing Mode. The instruction JMP address causes an unconditional jump.

Assembly Code:

```
ORG 100h

JMP TargetLabel

RET

TargetLabel:
MOV AX, 5678h
RET
```

Output:

The screenshot shows an x86 emulator window titled "emulator: noname.com_". The assembly code is displayed in a list with line numbers 01 through 09. The code is as follows:

```
01 ORG 100h
02
03 JMP TargetLabel
04
05 RET
06
07 TargetLabel:
08 MOV AX, 5678h
09 RET
```

Below the code, the emulator's control panel includes a menu bar (file, math, debug, view, external, virtual devices, virtual drive, help) and a toolbar with buttons for Load, reload, step back, single step, run, and a step delay slider set to 0 ms.

The registers window shows the following values:

Register	H	L
AX	56	78
BX	00	00
CX	00	07
DX	00	00
CS	F400	
IP	0154	
SS	0700	
SP	FFFA	
BP	0000	
SI	0000	

The memory window shows the current instruction at address F4154:0154, which is `CF 207 ±`. The BIOS memory window shows the following instructions:

```
BIOS DI
INT 020h
IRET
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD BH, BH
DEC BP
SBB CL, BH
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD BH, BH
DEC BP
```

The program jumps directly to TargetLabel and sets AX to 5678h.

Conclusion

We successfully redesigned an Instruction Set Architecture (ISA) for better efficiency and performance. We defined a 64-bit instruction format with optimized field sizes, introduced appropriate addressing modes, and implemented basic processor operations in assembly language. By running these programs on EMU8086, we verified the correctness of each operation. This provided practical experience in instruction set design, assembly programming, and understanding CPU-level performance improvements.